

存储性能分析程序

```
long data[MAXELEMS]; /* Global array to traverse */

/* test - Iterate over first "elems" elements of
 * array "data" with stride of "stride", using
 * using 4x4 loop unrolling.
 */
int test(int elems, int stride) {
    long i, sx2=stride*2, sx3=stride*3, sx4=stride*4;
    long acc0 = 0, acc1 = 0, acc2 = 0, acc3 = 0;
    long length = elems, limit = length - sx4;

    /* Combine 4 elements at a time */
    for (i = 0; i < limit; i += sx4) {
        acc0 = acc0 + data[i];
        acc1 = acc1 + data[i+stride];
        acc2 = acc2 + data[i+sx2];
        acc3 = acc3 + data[i+sx3];
    }

    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc0 = acc0 + data[i];
    }
    return ((acc0 + acc1) + (acc2 + acc3));
}
```

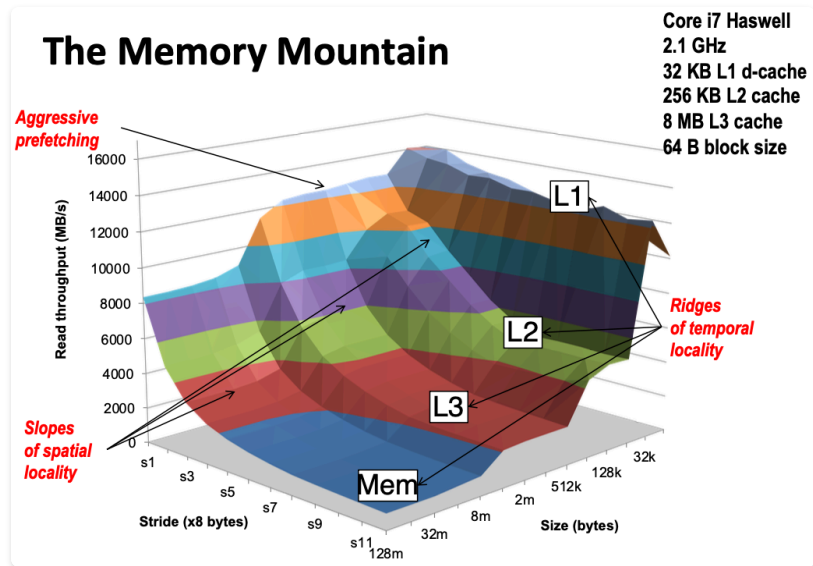
mountain/mountain.c

一个程序从存储系统中读数据的速率称为**读吞吐量** (read throughput), 或者有时称为 **读带宽** (read bandwidth)

如果一个程序在s秒的时间段内读n个字节, 那么这段时间内的读吞吐量就等于 n/s , 通常以兆字节每秒 (MB/s) 为单位

- test 存储性能分析函数:
 - 在 elems 限定范围内, 以 stride 为步长访问数组
 - 使用4路循环展开
 - 用不同的 size 和 stride 测试
 - 测试前先调用一次test()来使缓存热身
 - 计算访问时间, 得到吞吐量

存储器山 (Memory Mountain)



- 横轴: 访问数据块的大小 (Working Set Size)
 - 数据块越小, 时间局部性越好
 - 数据块越大, 就需要越低级的缓存来存储数据
 - 对应图中的L1, L2, L3, Mem
- 纵轴: 访问数据的步长 (Stride)
 - 步长越大, 跨越的缓存行越多, 空间局部性越差
- 颜色: 读吞吐量 (Read Throughput)等高线

存储器山呈阶梯形分布, 反映了多级缓存的层次结构
当数据块大小超过某一级缓存容量时, 吞吐量会骤降
在同一级缓存内, 访问步长越大, 吞吐量越低

- 特殊优化: 硬件预取 (aggressive prefetching)
 - 自动识别步长为1的顺序访问模式

可以看到, 吞吐量由时间局部性和空间局部性共同决定

矩阵访问的两种方式

矩阵数组在c语言中按行优先 (row-major) 存储,每一行的元素在内存中是连续存放的
对于较大的矩阵, 往往无法全部放入缓存, 需要多次读取缓存块到缓存中
每个缓存块将连续存放的若干元素一起装入缓存

在“一行内按列移动” (stride=1, 空间局部性好)

```
for (int row = 0; row < N; row++)  
    for (int col = 0; col < N; col++)  
        sum += a[row][col];    // 固定行row, 沿行向右走
```

- 访问相邻元素, 空间局部性强
- 若缓存块大小为 B 字节, 元素大小为 sizeof(a_ij)
 每装入一块可复用 B/sizeof(a_ij) 个元素
- 近似缺失率:
 $\text{miss_rate} \approx \text{sizeof(a_ij)} / B$

在“一列内按行移动” (stride=N, 几乎无空间局部性)

```
for (int col = 0; col < N; col++)  
    for (int row = 0; row < N; row++)  
        sum += a[row][col];    // 固定列col, 沿列向下走
```

- 每次访问跨过整行
 相邻两次访问通常落在不同的缓存块
- 几乎没有空间局部性
 前一次装入的缓存块很难被下一次复用
 每一次访问都导致一次缓存缺失
- 近似缺失率: $\text{miss_rate} \approx 1$

矩阵乘法中的循环次序

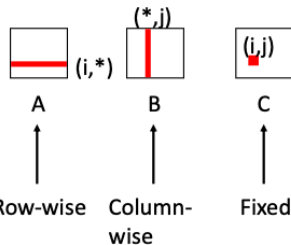
$$C[i][j] += A[i][k] * B[k][j]$$

- ijk (循环次序 $i \rightarrow j \rightarrow k$):
 - A: k 为内层, $A[i][k]$ 行访问, 空间局部性好
 - B: k 为内层, $B[k][j]$ 列访问, 空间局部性差
 - C: j 为内层, $C[i][j]$ 行访问, 空间局部性好
 - 内层循环中 C_{ij} 固定, 时间局部性好

```
/* ijk */
for (i=0; i<n; i++) {
  for (j=0; j<n; j++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

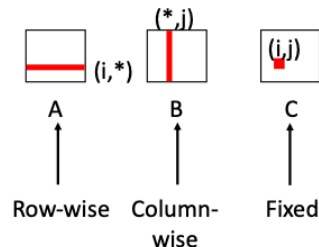
<u>A</u>	<u>B</u>	<u>C</u>
0.25	1.0	0.0

- jik (循环次序 $j \rightarrow i \rightarrow k$):
 - A: k 为内层, $A[i][k]$ 行访问, 空间局部性好
 - B: k 为内层, $B[k][j]$ 列访问, 空间局部性差
 - C: j 为外层, $C[i][j]$ 列访问, 空间局部性差
 - 内层循环中 C_{ij} 固定, 时间局部性好

```
/* jik */
for (j=0; j<n; j++) {
  for (i=0; i<n; i++) {
    sum = 0.0;
    for (k=0; k<n; k++)
      sum += a[i][k] * b[k][j];
    c[i][j] = sum;
  }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.25	1.0	0.0

矩阵乘法中的循环次序

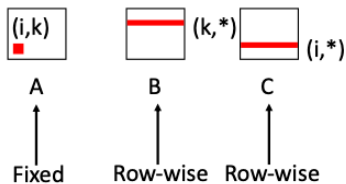
$$C[i][j] += A[i][k] * B[k][j]$$

- kij (循环次序 $k \rightarrow i \rightarrow j$):
- A: k 为外层, $A[i][k]$ 列访问, 空间局部性差
 - 内层循环中 A_{ij} 固定, 时间局部性好
- B: j 为内层, $B[k][j]$ 行访问, 空间局部性好
- C: j 为内层, $C[i][j]$ 行访问, 空间局部性好

```
/* kij */
for (k=0; k<n; k++) {
  for (i=0; i<n; i++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

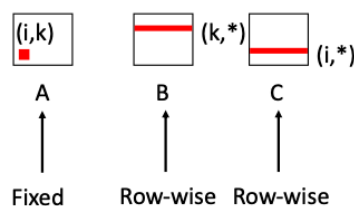
<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

- ikj (循环次序 $i \rightarrow k \rightarrow j$):
- A: k 为内层, $A[i][k]$ 行访问, 空间局部性好
 - 内层循环中 A_{ij} 固定, 时间局部性好
- B: j 为内层, $B[k][j]$ 行访问, 空间局部性好
- C: j 为内层, $C[i][j]$ 行访问, 空间局部性好

```
/* ikj */
for (i=0; i<n; i++) {
  for (k=0; k<n; k++) {
    r = a[i][k];
    for (j=0; j<n; j++)
      c[i][j] += r * b[k][j];
  }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

<u>A</u>	<u>B</u>	<u>C</u>
0.0	0.25	0.25

矩阵乘法中的循环次序

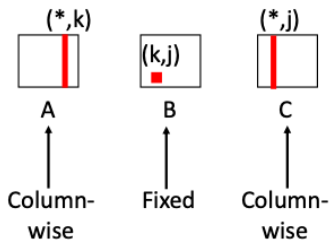
$$C[i][j] += A[i][k] * B[k][j]$$

- jki (循环次序 $j \rightarrow k \rightarrow i$):
- A: k 为外层, $A[i][k]$ 列访问, 空间局部性差
- B: j 为外层, $B[k][j]$ 列访问, 空间局部性差
 - 内层循环中 $B[k][j]$ 固定, 时间局部性好
- C: j 为外层, $C[i][j]$ 列访问, 空间局部性差

```
/* jki */
for (j=0; j<n; j++) {
  for (k=0; k<n; k++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```

matmult/mm.c

Inner loop:



Misses per inner loop iteration:

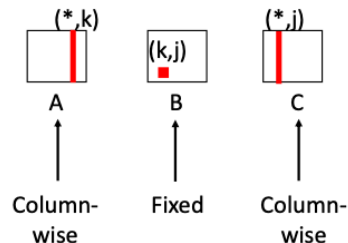
<u>A</u>	<u>B</u>	<u>C</u>
1.0	0.0	1.0

- kji (循环次序 $k \rightarrow j \rightarrow i$):
- A: k 为外层, $A[i][k]$ 列访问, 空间局部性差
- B: j 为内层, $B[k][j]$ 行访问, 空间局部性好
 - 内层循环中 $B[k][j]$ 固定, 时间局部性好
- C: j 为外层, $C[i][j]$ 列访问, 空间局部性差

```
/* kji */
for (k=0; k<n; k++) {
  for (j=0; j<n; j++) {
    r = b[k][j];
    for (i=0; i<n; i++)
      c[i][j] += a[i][k] * r;
  }
}
```

matmult/mm.c

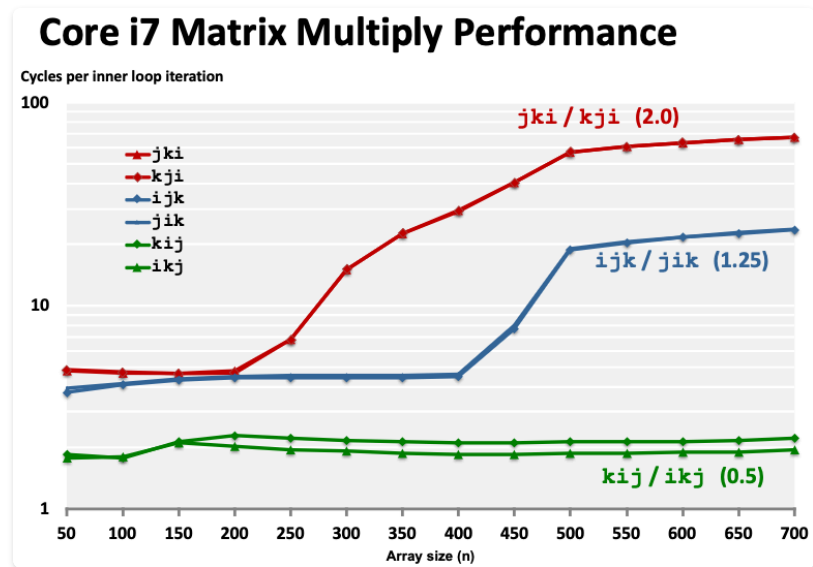
Inner loop:



Misses per inner loop iteration:

<u>A</u>	<u>B</u>	<u>C</u>
1.0	0.0	1.0

矩阵乘法中的循环次序



6种循环次序的性能比较
性能接近的次序被分为一类

- 三类循环次序中:
- jki/kji: 只对B有较好的局部性
- ijk/jik: 对A和C有较好的局部性
- kij/ikj: A、B、C均有较好的局部性
- 这符合我们的预期:
 - 循环次序的局部性越好, cache的表现越好

miss次数的计算方法

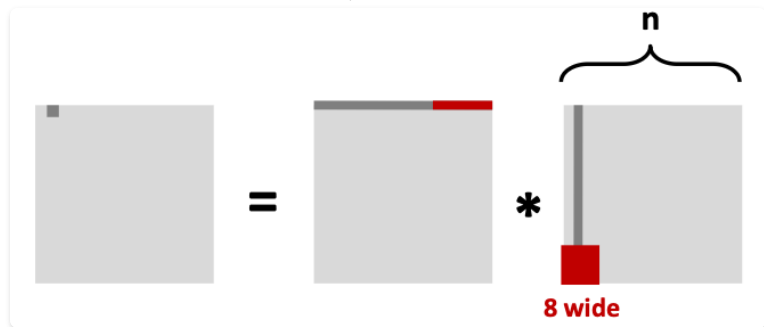
假设: $n \times n$ 矩阵元素为double类型(8字节), 缓存块大小为64字节(8个double)

缓存总大小 $C \ll n$, 即缓存无法存放矩阵中的一行数据

```
c = (double *) calloc(sizeof(double), n*n);

/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
            for (k = 0; k < n; k++)
                c[i*n + j] += a[i*n + k] * b[k*n + j];
}
```

每次内层循环时,缓存的运作方式如下



- 可以看到在每个 C_{ij} 的计算中:
- 对矩阵A的访问, 几乎每次内层循环都能利用上次加载的缓存块
 - 按行访问, 空间局部性好
 - miss次数为 $\frac{n}{8}$ (每8个元素产生1次miss)
- 对矩阵B的访问, 每次内层循环都需要加载新的缓存块
 - 按列访问, 空间局部性差
 - miss次数为 n (每个元素都产生1次miss)
- 故A、B总的miss次数为:

$$\text{Total misses} = n^2 \times \left(\frac{n}{8} + n \right) = \frac{9n^3}{8}$$

- C按行写入, miss次数为: $\frac{n^2}{8} \ll \frac{9n^3}{8}$

blocking (分块) 技术

假设:矩阵元素为double类型, 缓存总大小 $C \ll n$, 但足以存放3个 $x \times x$ 的子矩阵, 即 $3x^2 < C$

```
c = (double *) calloc(sizeof(double), n*n);

/* Multiply n x n matrices a and b */
void mmm(double *a, double *b, double *c, int n) {
    int i, j, k;
    for (i = 0; i < n; i+=B)
        for (j = 0; j < n; j+=B)
            for (k = 0; k < n; k+=B)
                /* B x B mini matrix multiplications */
                for (i1 = i; i1 < i+B; i1++)
                    for (j1 = j; j1 < j+B; j1++)
                        for (k1 = k; k1 < k+B; k1++)
                            c[i1*n + j1] += a[i1*n + k1]*b[k1*n + j1];
}
```

matmult/bmm.c

每次内层循环时,缓存的运作方式如下



- 将矩阵A, B, C划分为多个 $x \times x$ 的子矩阵
- 计算C的一个 $x \times x$ 子矩阵时:
 - 不断将A和B的一个子矩阵加载到缓存中
- 于是在计算C的这个子矩阵的过程中:
 - 对矩阵A、B、C的访问几乎都能命中缓存
 - 每个子矩阵miss次数为: $\frac{x^2}{8}$
 - A、B一共需要加载 $\frac{2*n}{x}$ 个子矩阵
- 故A、B总的miss次数为:

$$\text{Total misses} = \frac{n^2}{x^2} \times \frac{2 * n}{x} \times \frac{x^2}{8} = \frac{n^3}{4x}$$

- C的miss次数为: $\frac{n^2}{x^2} \times \frac{x^2}{8} = \frac{n^2}{8} \ll \frac{n^3}{4x}$
- 应该使用尽可能大的x值, 但 $3x^2 < C$, 即 $x < \sqrt{C/3}$

Cache 总结

Cache 的用途:

- **弥合CPU与主存的速度鸿沟**
 - CPU 的速度远超主存, Cache 能有效减少 CPU 的等待时间, 提升系统整体性能
- **利用局部性原理 (Principle of Locality)**
 - 程序行为具有局部性, 即倾向于在短期内访问邻近的数据 (**空间局部性**) 和重复访问邻近的数据 (**时间局部性**)。利用这一特性, Cache 将可能被访问的数据提前加载, 提高命中率
- **构建存储器层次结构 (Memory Hierarchy)**
 - Cache 是存储器层次结构的关键一环, 它在速度更快的寄存器和速度较慢的主存之间提供了一个折中的存储层, 实现了成本、容量和速度的平衡

如何用好 Cache:

- 编写对缓存友好的代码 (Cache-Friendly Code)
 - **关注程序的内层循环**
 - 大部分计算发生在内层循环, 优化内层循环的访存模式能带来最大的性能提升
- **最大化空间局部性**
 - 尽量按数据在内存中的存储顺序进行访问
stride = 1 的访问模式可以利用硬件预取机制
 - 按行优先顺序遍历数组
- **最大化时间局部性**
 - 一旦数据被读入缓存, 就应尽可能多地使用它
 - 使用分块 (blocking) 技术